

Bridge.py Setup Guide

Step-by-step instructions to run the LoRa Gateway USB Bridge on any Windows computer.

What this is

Your laptop runs **bridge.py**, a small Python program that reads LoRa packets coming in through the gateway ESP32 over USB and forwards them to your backend server. This guide explains exactly how to set that up on a fresh computer so anyone can clone the repo and run it.

System overview

Phone --(WiFi)--> Node ESP32 --(LoRa)--> Gateway ESP32 --(USB)--> Laptop running bridge.py --

The gateway has no WiFi. The laptop does all internet work. This makes the whole setup work on any network or even offline (if backend runs on the same laptop).

1. What you need

Hardware

- The Gateway ESP32 board (Heltec WiFi LoRa 32 V2) flashed with **gateway.ino**
- A USB cable that supports data (not charge-only)
- A Windows computer
- Internet access on first setup (for downloads only)

Software

- **Python 3.8+** (download from python.org)
- **pip** (comes with Python)
- **Silicon Labs CP210x USB-to-UART driver** (for the Heltec board)
- The **bridge.py** file from the project repository
- The backend server (running locally or on the same network)

Heads up

Heltec V2 boards use the CP2102 USB-to-UART chip. Some Windows installs already have the driver. If you plug in the board and no COM port appears in Device Manager, install the CP210x driver from Silicon Labs before continuing.

2. Install Python

Step 2.1 - Download

Go to <https://www.python.org/downloads/> and download the latest Python 3 installer for Windows.

Step 2.2 - Install with PATH enabled

Run the installer. On the first screen, **check the box that says "Add Python to PATH"**. This is the most important checkbox in the whole guide. Without it, the python command will not work in Command Prompt.

Then click Install Now.

Step 2.3 - Verify

Open a new Command Prompt (Win+R, type cmd, Enter) and run:

```
python --version
```

You should see something like Python 3.11.5. If you see *not recognized*, the PATH box was unchecked - reinstall and check it.

3. Install the two Python libraries

bridge.py uses **pyserial** to read the USB serial port and **requests** to POST to the backend. Install both in one go:

```
pip install pyserial requests
```

You should see lines like "Successfully installed pyserial-3.5 requests-2.32.x". If pip says it is not recognized, run `python -m pip install pyserial requests` instead.

4. Get bridge.py from the repository

Option A - Clone the whole repo with Git

```
git clone https://github.com/YOUR-USERNAME/YOUR-REPO.git
cd YOUR-REPO
```

Option B - Download just bridge.py

On the GitHub repo page, navigate to bridge.py, click the file, click the **Raw** button, then right-click anywhere on the page and choose **Save as....** Save it to a folder you can find easily, e.g. C:\Users\YourName\bridge.py.

Watch out

If you save through Notepad, set 'Save as type' to **All Files** or it becomes bridge.py.txt. Check with `dir bridge*` in Command Prompt.

5. The complete bridge.py code

For reference, here is the full content of bridge.py:

```
"""
LoRa Gateway USB Bridge
Reads packets from gateway.ino over USB serial,
parses them, and POSTs them to the backend API.
"""

import json, sys, time
import requests, serial

# ===== CONFIG =====
SERIAL_PORT = "COM4"
BAUD_RATE   = 115200
API_URL     = "http://localhost:8000/api/users/entry"
# =====

def parse_packet(line: str):
    if not line.startswith("PACKET:"):
        return None
    body = line[len("PACKET:"):].strip()
    fields = {}
    for part in body.split(","):
        if ":" not in part:
            continue
        key, value = part.split(":", 1)
        fields[key.strip().upper()] = value.strip()
    try:
        return {
            "name": fields.get("NAME", ""),
            "emergency_number": fields.get("PHONE", ""),
            "latitude": float(fields.get("LAT", "0")),
            "longitude": float(fields.get("LON", "0")),
        }
    except ValueError as exc:
        print(f" ! parse error: {exc}")
        return None

def post_to_backend(payload):
    try:
        print(f" -> POSTing to {API_URL}")
        print(f"      payload: {json.dumps(payload)}")
        r = requests.post(API_URL, json=payload, timeout=10,
                          headers={"Content-Type": "application/json"})
        print(f" <- HTTP {r.status_code}: {r.text[:200]}")
    except requests.exceptions.RequestException as exc:
        print(f" ! POST failed: {exc}")

def main():
    print(f"Opening {SERIAL_PORT} at {BAUD_RATE} baud...")
    try:
        ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
    except serial.SerialException as exc:
        print(f"FATAL: cannot open {SERIAL_PORT}: {exc}")
```

```

        sys.exit(1)
    print("Listening for LoRa packets...")
    print("=" * 60)
    while True:
        try:
            raw = ser.readline()
            if not raw: continue
            line = raw.decode("utf-8", errors="replace").strip()
            if not line: continue
            print(f"[gateway] {line}")
            if line.startswith("PACKET:"):
                p = parse_packet(line)
                if p: post_to_backend(p)
        except KeyboardInterrupt:
            break
        except Exception as exc:
            print(f" ! error: {exc}"); time.sleep(1)
    ser.close()

if __name__ == "__main__":
    main()

```

6. Configure bridge.py for this computer

Two settings at the top of bridge.py almost always need to be adjusted when moving to a new machine: SERIAL_PORT and API_URL.

Step 6.1 - Find the gateway's COM port

Plug the gateway board into USB. Open **Device Manager** (right-click Start → Device Manager). Expand **Ports (COM & LPT)**. You should see *Silicon Labs CP210x USB to UART Bridge (COM5)* or similar. Note the COM number.

Step 6.2 - Update SERIAL_PORT

Open bridge.py in a text editor and change:

```
SERIAL_PORT = "COM4"    # <-- change to your COM port
```

Step 6.3 - Update API_URL

Decide where the backend will run:

- If backend runs on this same laptop → leave localhost
- If backend runs on another machine in the network → use that machine's LAN IP, e.g.:

```
API_URL = "http://192.168.1.50:8000/api/users/entry"
```

Tip

To find a laptop's LAN IP, run `ipconfig` in Command Prompt and look at the WiFi adapter's IPv4 Address.

7. Make sure the backend is running

bridge.py is just a pipe - it cannot save data on its own. The backend must already be listening at the URL set in API_URL before you submit any alerts.

How to start the backend depends on the stack. Common cases:

```
# Node.js / Express / Next.js
cd backend-folder
npm install
npm run dev
```

```
# Python FastAPI / Flask
cd backend-folder
pip install -r requirements.txt
uvicorn main:app --port 8000
```

Verify by opening **http://localhost:8000** in a browser. You should see the admin page or at least a non-error response.

8. Run bridge.py

Step 8.1 - Close conflicting programs

Make sure the **Arduino IDE Serial Monitor** is closed - only one program can read a COM port at a time. If it is open, bridge.py will fail with "Access denied".

Step 8.2 - Start it

Open Command Prompt, navigate to the folder containing bridge.py, and run:

```
cd C:\Users\YourName
python bridge.py
```

Step 8.3 - Expected output

Within 1-2 seconds you should see the gateway boot up:

```
Opening COM5 at 115200 baud...
Listening for LoRa packets from gateway...
=====
[gateway] ==== GATEWAY BOOT (USB bridge mode) ====
[gateway] LoRa.begin OK
[gateway] ==== GATEWAY READY ====
[gateway] [heartbeat] uptime=10s
```

If you see heartbeats every 10 seconds, everything is working. If you do not see them, press the **RST** button on the gateway board - the boot logs will appear in the bridge window.

Step 8.4 - End-to-end test

Connect a phone to the EmergencyNode1 WiFi, open the captive portal (or browse to <http://192.168.4.1>), fill in name and number, tap SEND. The bridge window should print:

```
[gateway] PACKET:NAME:John,PHONE:0300...,LAT:24.91,LON:67.11
-> POSTing to http://localhost:8000/api/users/entry
<- HTTP 200: {...}
[gateway] META: size=58 RSSI=-32 SNR=10.5
```

And the admin page should update with the new entry.

9. Troubleshooting

"python is not recognized"

Python is not on PATH. Reinstall Python and check "Add to PATH", or use `py bridge.py` instead.

"Cannot find file bridge.py"

You are not in the folder where `bridge.py` lives. Run `dir bridge*` to confirm. If you see `bridge.py.txt`, rename with `ren bridge.py.txt bridge.py`.

"could not open COMx" / "Access is denied"

Close the Arduino Serial Monitor. Make sure no other program has the port open. Re-check the COM number in Device Manager.

No gateway output, just "Listening for LoRa packets..."

Press RST on the gateway board. If still nothing, the COM port is wrong or the gateway is unflashed/dead. Re-upload `gateway.ino`.

"POST failed: ConnectTimeout"

Backend is not running, or `API_URL` is wrong, or a firewall is blocking. Open `http://localhost:8000` in a browser to verify the backend is up.

RSSI very low (e.g. -120 dBm)

Nodes are too far away or have antenna issues. Move closer or attach the antenna that came with the Heltec board.

Captive portal does not open on phone

Open a browser manually and go to `http://192.168.4.1`. Also try forgetting the WiFi network and reconnecting. Some Android versions show captive portal as a notification - swipe down and tap it.

10. Quick reference card

Once everything is set up, the daily workflow is:

Step	Action
1	Plug gateway ESP32 into USB
2	Start backend server
3	Run <code>python bridge.py</code>
4	Power on the nodes
5	Users submit alerts -> admin page updates

End of guide. If something breaks that is not covered here, open an issue on the project repository.